Лабораторна робота 2

Робота зі вказівниками, рядками та файлами

1 Завдання на лабораторну роботу

1.1 Сума мінімального і максимального елементів

Написати програму, яка обчислює суму мінімального і максимального елементів масиву значень з рухомою комою подвійної точності. Здійснити пошук мінімального і максимального елементів у двох окремих функціях. Програма повинна задовольняти такі вимоги:

- довжину масиву слід прочитати з клавіатури за допомогою функції `scanf()`;
- масив створюється в динамічній пам'яті;
- масив повинен заповнюватися випадковими значеннями в діапазоні від 0 до 100;
- обхід масивів повинен здійснюватися за допомогою адресної арифметики;
- параметрами функцій обчислення мінімального і максимального елементів є вказівники на `double` і розміри масиву;
- результат слід вивести за допомогою функції `printf()`;
- необхідно забезпечити звільнення пам'яті.

1.2 Перевірка паліндромів

Прочитати з клавіатури речення (масив символів) за допомогою функції `getline()`, перевірити, чи є воно паліндромом і вивести відповідне повідомлення. Рекомендація: під час введення не використовувати великих літер. Результат вивести за допомогою функції `printf()`.

Паліндром – це речення, що однаково читається в обох напрямках (зліва направо та справа наліво), наприклад, "аби ріці риба". При цьому пропуски та розділові знаки не враховуються. В програмній реалізації їх доцільно видалити та здійснювати перевірку отриманого рядка.

1.3 Індивідуальне завдання

Підготувати текстовий файл, який містить інформацію про розміри двовимірного масиву та самі елементи масиву. Розміри у файлі повинні бути представлені двома цілими значеннями, розділеними пропуском. На наступних рядках розташувати елементи масиву рядок за рядком.

У програмі виконати такі дії:

- Створити в динамічній пам'яті двовимірний масив цілих елементів, прочитавши його розміри з файлу.
- Прочитати елементи масиву з файлу рядок за рядком.
- Створити в динамічній пам'яті другий масив необхідних розмірів.
- Обробити перший масив та заповнити другий відповідно до індивідуального завдання попередньої лабораторної роботи.
- Створити інший текстовий файл і записати в нього елементи другого масиву.

- Звільнити пам'ять, яку займали масиви.

Паралельно з виведенням у файл слід забезпечити виведення результатів у консольне вікно за допомогою функції `printf()`. Під час роботи з одновимірним масивом замість роботи з індексом застосувати адресну арифметику.

2 Методичні вказівки

2.1 Визначення вказівників

Кожна змінна знаходиться в унікальному місці в пам'яті комп'ютера і має свою адресу. *Вказівник* є змінною, яка містить адресу пам'яті. Для визначення вказівника використовують зірочку, яка передує імені змінної. В наведеному нижче прикладі змінна `p` може зберігати адресу цілої змінної:

```
int *p;
```

Вказівник, значення якого дорівнює нулю, має назву *нульового вказівника* (null pointer). Всі вказівники, коли вони створюються, одразу або пізніше повинні бути ініціалізовані. Якщо певне значення поки не можна визначити, вказівнику доцільно присвоїти нуль. Вказівник, який не ініціалізовано, називається *диким вказівником* (wild pointer). Дикі вказівники дуже небезпечні.

Щоб присвоїти значення або ініціалізувати вказівник, перед ім'ям змінної, адреса присвоюється вказівнику, розташовують *оператор отримання адреси* (`&`). Наприклад:

```
int i = 5;
int *p = &i;
```

Щоб отримати доступ до даних, що зберігаються за адресою, можна використовувати так зване *розіменування* (dereferencing). Для розіменування вказівника оператор розіменування (`*`) встановлюється перед ім'ям вказівника. Наприклад:

```
int j = *p + i; // j = 5 + 5 = 10
```

Фактично `*p` є синонімом `i`. З `*p` можна працювати як зі звичайною цілою змінною.

```
*p = 6; // i = 6
```

Можна створити масив указівників:

```
int i = 0;
int j = 1;
int k = 10;
int *pa[3];
pa[0] = &i;
pa[1] = &j;
pa[2] = &k;
for (int i = 0; i < 3; i++)
```

```
{
    cout << *pa[i] << ' '; // 0 1 10
}
```

Можна здійснити ініціалізацію масиву вказівників адресами змінних:

```
int *pb[] = {&i, &j, &k};
for (int i = 0; i < 3; i++)
{
    cout << *pb[i] << ' '; // 0 1 10
}
```

Правила сумісності типів для вказівників є більш жорсткими, ніж аналогічні правила для значень. Можна присвоїти ціле значення дійсному і навпаки, але не можна присвоювати вказівники різних типів один одному:

```
int k = 1;
double d = k;
int j;
j = d;
int *pk = &k;
double *pd = &d;
int *pj;
pj = pk;         // OK. Два вказівника вказують на одну змінну
pd = pk;         // Синтаксична помилка
pj = pd;         // Синтаксична помилка
float *pf = pd;  // Синтаксична помилка
```

Вказівникам можна присвоювати ціле значення 0, або значення константи `nullptr` (починаючи з версії C ++11) .

Існує особливий тип вказівників: `void*`. Можна призначити вказівники будь-яких типів до `void*`. Змінні типу `void*` використовуються для зберігання вказівників різних типів. У наведеному нижче прикладі, масив `void*` містить адреси різних змінних:

```
int i = 0;
double d = 1.5;
char c = 'A';
void *p[4] = {&i, &d, &c}; // p[3] = 0
double *pd = &d;
p[3] = pd;
```

Не можна розіменовувати `void*`. Для того, щоб отримати вказівники конкретних типів, слід привести вказівники до необхідних типів за допомогою оператору `static_cast`:

```
char *pc = static_cast<char*>(p[2]);
cout << *pc; // 'A'
```

Можна створювати вказівники на константний об'єкт: значення, на яке вказує вказівник, не може бути змінено, але сам вказівник може бути переміщений на іншу змінну або константу.

```
int k = 4;
const int *pk = &k; // Вказівник на константу
k = 5;
cout << *pk;        // 5
*pk = 6;            // Помилка!
```

Константний вказівник (ключове слово `const` стоїть після `*`) – це вказівник, який не може бути змінений. Проте, значення, на яке вказує вказівник може бути змінене. Константні вказівники обов'язково повинні бути ініціалізовані:

```
int i = 1;
int * const cp = &i;
cout << *cp;         // 1
int j = 2;
cp = &j;             // Помилка!
```

Можна також створювати константні вказівники на константні об'єкти.

Аргументи функції можуть бути оголошені як вказівники. Це дозволяє надсилати адреси заданих змінних в функцію і змінювати ці змінні, використовуючи їх адреси. Припустимо, що нам потрібно поміняти місцями значення двох цілих змінних. Раніше розглядався приклад функції з використанням посилань. Функція може також використовувати вказівники як параметри:

```
void swap(int *p1, int *p2)
{
    int x = *p1;
    *p1 = *p2;
    *p2 = x;
}
```

Тепер функцію `swap()` можна викликати з функції `main()`:

```
void main()
{
    int a = 1;
```

```
    int b = 2;
    swap(&a, &b); // Отримуємо адреси змінних
    cout << a << endl; // 2
    cout << b << endl; // 1
}
```

2.2 Зв'язок масивів зі вказівниками

У C++ , ім'я масиву є постійним вказівником на перший елемент масиву. Таким чином, у визначенні

```
int a[50];
```

`a` – це вказівник на `&a[0]`, тобто на початковий елемент масиву. Можна використовувати імена масивів як константні вказівники і навпаки:

```
int a[5] = {1, 2, 4, 8, 16};
int *p = a;
cout << *a << endl;   // 1
cout << p[2] << endl; // 4
```

До вказівників можна застосувати деякі операції. Арифметика вказівників обмежується додаванням, відніманням і порівнянням. Під час виконання арифметичних операцій з вказівниками передбачається, що вказівники вказують на масив об'єктів. Додавши ціле значення до вказівника, ми переміщуємо його на відповідне число об'єктів у масиві. Якщо, наприклад, тип має розмір 10 байтів, а потім ми додали ціле число 5, вказівник переміщується на 50 байтів в пам'яті. Можна застосувати інкремент і декремент, а також операції складеного присвоювання для неконстантних вказівників.

```
int a[5] = { 1, 2, 4, 8, 16 };
int *p = a;               // p вказує на a[0]
cout << *(a + 3) << endl; // 8
p++;                      // p вказує на a[1]
cout << p << endl;        // 2
p += 3;                   // p вказує на a[4]
cout << p << endl;        // 16
a++;                      // Помилка! a є константним вказівником
```

Різниця між двома вказівниками на різні елементи масиву повертає кількість елементів, які розташовані між цими вказівниками (включаючи перший і не включаючи останній). Наприклад,

```
int a[5] = { 1, 2, 4, 8, 16 };
int *p1 = a;               // p1 вказує на a[0]
int *p2 = a + 3;           // p2 вказує на a[3]
```

```
cout << p2 - p1            // 3;
```

Різниця між двома вказівниками має сенс тільки тоді, коли обидва вказівники вказують на елементи одного масиву.

Перевірка виходу за межі масиву не здійснюється. У наведеному нижче прикладі `p` вказує на елемент, який не існує:

```
int a[5] = {1, 2, 4, 8, 16};
int *p3 = a + 5;           // Такого елемента немає
```

Розіменування `p3` є небезпечним.

Вказівники можна використовувати для опису параметра функції типу масиву. Вихідний масив може бути змінений всередині функції:

```
void modifyStartingElement(int *p)
{
    p[0] = 0;
}

void main()
{
    int a[] = {1, 2, 3};
    modifyStartingElement(a);
    for (int i = 0; i < 3; i++)
    {
        cout << a[i] << ' '; // 0 2 3
    }
}
```

Завдяки вказівникам можна передавати багатовимірні масиви як параметри. Але для забезпечення гнучкості слід описувати такі параметри як вказівники на дані (одновимірний масив). Під час виклику функції необхідно явно перетворювати тип. Наприклад, наведена нижче функція може бути використана для обчислення суми всіх елементів двовимірного масиву з `m` рядків і `n` стовпців. Реалізація функції передбачає обчислення індексу з урахуванням розташування елементів двовимірного масиву:

```
double sum(double* a, int m, int n)
{
    double result = 0;
    for (int i = 0; i < m; i++)
    {
```

```
        for (int j = 0; j < n; j++)
        {
            result += a[i * n + j];
        }
    }
    return result;
}

int main()
{
    double arr[][3] = { { 1, 2, 3 },
                        { 4, 5, 6 } };
    cout << sum((double*) arr, 2, 3) << endl; // явне перетворення типів
    return 0;
}
```

2.3 Використання динамічної пам'яті

Під час запуску програми операційна система налаштовує різні області пам'яті:

- Глобальні змінні знаходяться в *глобальному просторі імен*, для якого виділяється спеціальний сегмент пам'яті.
- *Регістри* утворюють особливу область пам'яті, вбудовану в центральний процесор.
- *Стек* (stack, стек викликів) це спеціальна область пам'яті, виділена для зберігання даних окремих функцій. Ознакою стеків як структур даних є принцип *last-in, first-out* (LIFO, першим зайшов, останнім вийшов).
- Решта пам'яті, розподіленої для програми, – це так звана *динамічна пам'ять* (free store).

Програмісти можуть використовувати динамічну пам'ять для контрольованого виділення пам'яті й звільнення змінних з пам'яті. Оператор `new` використовують для розташування змінних у динамічній пам'яті. Операція `new` повертає вказівник на об'єкт розташований в динамічній пам'яті.

```
int *p = new int;
*p = 65;
. . .// *p можна вільно використовувати, поки пам'ять не буде звільнена
```

Змінна може бути ініціалізована початковим значенням:

```
int *p = new int(65); // *p = 65
```

Змінна, яка була створена в динамічній пам'яті, повинна бути звільнена за допомогою оператору `delete`:

```
delete p;
```

Після звільнення пам'яті, пам'ять, що раніше була виділена для змінної `*p` може бути використана для розташування інших змінних.

Якщо після імені типу в операції `new` розташувати квадратні дужки з цілим значенням всередині, в динамічній пам'яті можна розмістити масив відповідного типу. Ціле значення в квадратних дужках – це кількість елементів, для визначення якої можна використовувати будь-які вирази або змінні, що приводяться до `int`. Після створення, використання динамічних масивів і звичайних масивів практично не відрізняються:

```
int n;
cin >> n;
double *pa = new double[n];
for (int i = 0; i < n; i++)
{
    pa[i] = 0;
}
. . .
```

Динамічну пам'ять, яка була виділена для масиву, звільняють в такий спосіб:

```
delete [] pa;
```

Необхідно стежити, щоб уся пам'ять, виділена за допомогою операції `new`, була звільнена за допомогою операції `delete`.

Якщо в динамічній пам'яті створити масив вказівників, а потім створити кілька одновимірних масивів і записати їх адреси в масив вказівників, ми отримаємо масив масивів, з яким можна працювати як зі звичайним двовимірним масивом:

```
cin >> m >> n;
double** a = new double*[m];
for (int i = 0; i < m; i++)
{
    a[i] = new double[n];
}
for (int i = 0; i < m; i++)
{
    for (int j = 0; j < n; j++)
    {
        a[i][j] << i + j;
    }
```

```
}
```

Дуже важливо звільнити пам'ять в правильному порядку:

```
for (int i = 0; i < m; i++)
{
    delete[] a[i];
}
delete[] a;
```

Аналогічно можна створювати масиви з більшою кількістю розмірностей.

Недоліком багатовимірних масивів в динамічній пам'яті є певне зниження ефективності під час роботи, але є істотні переваги:

- Можна отримувати (обчислювати) необхідну кількість рядків і стовпців під час виконання програми.
- Можна створювати масиви з різною довжиною рядків, наприклад, можна створити масив для зберігання трикутної матриці.
- Можна створювати функції для роботи з двовимірними масивами.

Останню перевагу можна пояснити на такому прикладі. Функція `fill()` записує вказане значення в усі елементи масиву:

```
#include <iostream>
using namespace std;

void fill(double **arr, int m, int n, double value) {
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
            arr[i][j] = value;
        }
    }
}

int main()
{
    int m, n;
    cin >> m >> n;
    double** a = new double*[m];
```

```
    for (int i = 0; i < m; i++)
    {
        a[i] = new double[n];
    }
    fill(a, m, n, 10);
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
            cout << a[i][j] << " ";
        }
        cout << endl;
    }
    for (int i = 0; i < m; i++)
    {
        delete[] a[i];
    }
    delete[] a;
}
```

Слід пам'ятати, що таку функцію можна застосовувати лише для двовимірних масивів у динамічній пам'яті. Останній рядок коду спричинить помилку компілятору:

```
double b[3][4];
fill(b, 3, 4, 10); // помилка компіляції
```

2.4 Масиви символів

У мові C++ (як і в C) рядок – це масив символів, що закінчуються нульовим символом (символ з кодом 0). Можна створити й ініціалізувати рядок так само як і будь-який інший масив. Наприклад,

```
char name[] = {'A', 'n', 'd', 'r', 'e', 'w', '\0'};
```

Останній символ `'\0'` використовують як ознаку закінчення рядка. Рядок може бути ініціалізований літералом:

```
char name[] = "Andrew";
```

Додавати нульовий символ вручну не треба, оскільки компілятор додає його автоматично. Таким чином, цей масив має сім елементів.

Мова C++ дозволяє читати масив символів без циклу. Важливо переконатися, що є достатньо пам'яті для розташування символів:

```
char s[30];
cin >> s;
```

Вказівники на символ можна ініціалізувати літералами:

```
char *st = "C++";
```

Компілятор виділяє пам'ять для зберігання чотирьох символів (включаючи `'\0'`) і записує адресу початкового символу в `st`.

Стандартна бібліотека C пропонує набір функцій для обробки рядків з завершальним нульовим символом. Наприклад,

```
// Повертає довжину рядка (без '\0'):
int strlen(const char *s);


// Порівнює два рядки і повертає від'ємне значення
// якщо s1 менше ніж s2, нуль, якщо вони однакові,
// і додатне значення в іншому випадку:
int strcmp(const char *s1, const char *s2);


// копіює s2 в s1:
strcpy(char *s1, const char *s2);
```

Функція `strcmp()` реалізує порівняння рядків за абеткою. Для використання наведених функцій слід підключити заголовний файл `<cstring>`. Оголошення функцій в цьому файлі розташовані в просторі імен `std`.

2.5 Використання виведення та введення у стилі мови C

Мова C++ дозволяє використовувати функції виведення і введення, успадкованих у мови C. Для роботи з цими функціями слід підключити заголовний файл `stdio.h`. Необхідні функції оголошені у глобальній області видимості.

Примітка. Більш сучасний варіант заголовного файлу – `cstdio`. Оголошення функцій в цьому файлі розташовані в просторі імен `std`.

Функцію `printf()` можна використовувати для виведення на консоль. Вона може бути викликана одним або декількома аргументами. Константний рядок може бути виведено на консоль, якщо вказати його як аргумент:

```
printf("Hello"); // так само, як cout << "Hello";
```

Якщо необхідно вивести числа, всередині першого аргументу вказують так звані *символи форматування*. Послідовність форматування починається з символу `%` з подальшим визначенням

розміру і безпосереднім символом форматування. Іноді встановлювати розмір поля не треба. Наприклад:

```
int k = 12;
printf("%i", k);
```

Найбільш важливими символами форматування є:

Символи	Дані, які виводяться
`%d` або `%i`	`int`
`%c`	окремий символ
`%e` або `%E`	`float` або `double` у форматі `[-]d.ddd e±dd` або `[-]d.ddd E ±dd`
`%f`	`float` або `double` у форматі `[-]ddd.ddd`
`%p`	адреса (вказівник)
`%s`	Рядок виведення
`%u`	`unsigned int`
`%x` або `%X`	`int` як шістнадцяткове число
`%%`	символ %

Символ "мінус" перед послідовністю форматування обумовлює вирівнювання по лівому краю.

Функція `scanf()` дозволяє зчитувати зі стандартного вхідного потоку. Перший параметр– це рядок форматування. Він дозволяє, зокрема, вказувати роздільники. Інші параметри – вказівники на змінні, значення яких повинні бути прочитані. Функція повертає кількість байтів, які читалися. Для читання даних типу `double` слід використовувати `lf` замість `f`. Наприклад:

```
int n;
float x;
double y;
scanf("%d, %f %lf", &n, &x, &y);
printf("%10d ", n);
```

```
printf("%10.5f \n", x);
printf("%-10.5f", y);
```

Під час введення перші два числа слід розділити комою.

На відміну від C++, мова C не підтримує роботу з потоками `cin` і `cout`. Тому найпростіша програма мовою C відрізнятиметься від наведеної раніше:

```
#include <stdio.h>


int main()
{
    printf("Hello, world!\n");
    return 0;
}
```

Примітка: варіант підключення `cstdio` неприйнятний для мови C, яка не підтримує просторів імен.

Недоліком функцій C в порівнянні з потоками C++ є те, що ці функції є небезпечними з точки зору типів. Це може призвести до численних помилок під час виконання, пов'язаних з невідповідністю типів змінних. Компілятор не може перевірити відповідність під час трансляції.

Більш небезпечною є функція `scanf()`. Некоректне використання цієї функції може призвести до зависання програми. Тому в сучасних версіях Visual Studio спроба скомпілювати програму, яка містить виклики `scanf()`, спричиняє виникнення помилки компіляції "`'scanf': This function or variable may be unsafe`". Для того, щоб скомпілювати таку програму, слід заборонити відповідний контроль, додавши першим рядком програми директиву препроцесору `#define _CRT_SECURE_NO_WARNINGS`. Наведена нижче програма здійснює читання цілого значення, збільшення його на одиницю й виведення результату на консоль:

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>


int main()
{
    int k;
    scanf("%d", &k);
    k++;
    printf("%d", k);
    return 0;
}
```

Існують також функції `getchar()` і `putchar()` для відповідно читання і виведення окремих символів.

2.6 Робота з файлами

Файлові потоки дозволяють здійснювати введення з текстових файлів і виведення в текстові файли. Файлові потоки забезпечують зручність роботи через те, що відкрити файл можна, створюючи об'єкт, і файл буде автоматично закритий перед знищенням потоку (через виклик так званого деструктора).

Класи файлових потоків `std::ifstream` і `std::ofstream` визначені в заголовному файлі `fstream`. Файловий потік повинен бути підключений до файлу, перш ніж він може бути використаний. У наведеному нижче прикладі програма зчитує ціле значення з файлу `"data.txt"` у змінну `k`. Це значення буде записано в інший файл:

```
#include <fstream>

. . .

int k;

std::ifstream inFile("data.txt");

inFile >> k;

std::ofstream outFile("result.txt");

outFile << k;

. . .
```

Якщо необхідно перевірити можливість читання даних з файлу, можна перевірити результат методу `eof()`. Можна також перетворити об'єкти потоку в цілі числа і в такий спосіб перевірити стан потоку. Другий підхід є більш ефективним.

Інший підхід – це використання вказівника на файл (`FILE *`) і пари функцій `fscanf` / `fprintf`. Такий підхід є небезпечним з точки зору типів.

3 Приклади програм

3.1 Максимальний елемент

Припустимо, необхідно створити програму, в якій здійснюється читання з текстового файлу кількості елементів масиву дійсних чисел, а також самі елементи. Масив створюється в динамічній пам'яті. Далі в масиві знаходиться максимальний елемент і виводиться на екран функцією `printf()`.

Спочатку в теці проєкту слід створити текстовий файл (`in.txt`), наприклад, з таким вмістом:

```
5

0.1 1.6 0.2 0.8 0.4
```

У файлі перший рядок – кількість елементів масиву, другий рядок – безпосередньо елементи.

У програмі замість індексів ми будемо використовувати вказівники та адресну арифметику. Програма буде такою:

```
#include <iostream>

#include <fstream>
```

```
#include <cstdio>

using std::ifstream;
using std::printf;

int main()
{
    ifstream in("in.txt"); // відкриваємо файл
    int n;
    in >> n; // читаємо з файлу кількість елементів масиву
    double* a = new double[n];
    for (double* p = a; p != a + n; p++)
    {
        in >> *p; // читаємо елементи
    }
    double* max = a; // спочатку max вказує на нульовий елемент
    for (double* p = a + 1; p != a + n; p++)
    {
        if (*p > *max)
        {
            max = p; // змінюємо значення вказівника
        }
    }
    printf("max item: %5.2f", *max);
    delete[] a;
    return 0;
}
```

3.2 Сума добутків

У наведеній нижче програмі двовимірний масив дійсних чисел заповнюється випадковими значеннями в діапазоні від 0 до 3 і здійснюється знаходження суми добутків його рядків. Кількість рядків (`m`) і стовпців (`n`) користувач уводить з клавіатури, для чого застоствано функцію `scanf()`.

Оскільки стандартні засоби C, оголошені в заголовному файлі `cstdlib`, не надають функції для о тримання дійсних псевдовипадкових чисел, її можна створити самостійно.

```
#define _CRT_SECURE_NO_WARNINGS
```

```
#include <cstdlib>
#include <cstdio>
#include <ctime>

using std::srand;
using std::time;
using std::rand;
using std::scanf;
using std::printf;

// Функція повертає псевдовипадкове число в діапазоні
// від 0 (включаючи) до 1 (не включаючи).
// RAND_MAX - ціла константа, максимальне значення,
// яке повертає rand()
double doubleRand() {
    // Додаємо 1.0 для того, щоб включити 0 і не включити 1:
    return rand() / (RAND_MAX + 1.0);
}

int main()
{
    srand(time(0));
    int m, n;
    // Введення даних і перевірка кількості прочитаних байтів:
    if ((scanf("%d %d", &m, &n)) < 2)
    {
        return -1;
    }
    // Створення і заповнення масиву:
    double** a = new double*[m];
    for (int i = 0; i < m; i++)
    {
        a[i] = new double[n];
        for (int j = 0; j < n; j++)
```

```
        {
            a[i][j] = doubleRand() * 3;
        }
    }
    // Виведення елементів масиву рядок за рядком:
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
            printf("a[%d][%d] = %f\t", i, j, a[i][j]);
        }
        printf("\n");
    }
    // Обчислення суми добутків:
    double sum = 0;
    for (int i = 0; i < m; i++)
    {
        double product = 1;
        for (int j = 0; j < n; j++)
        {
            product *= a[i][j];
        }
        sum += product;
    }
    printf("sum: %f", sum);
    // Звільнення пам'яті:
    for (int i = 0; i < m; i++)
    {
        delete[] a[i];
    }
    delete[] a;
    return 0;
}
```

3.3 Пошук від'ємних елементів

Припустимо, що ми маємо масив дійсних чисел. Програма повинна обчислити кількість від'ємних елементів і створити масив відповідного розміру. Від'ємні значення повинні бути поміщені в новий масив.

Сирцевий код функції `main()` буде таким:

```
const int n = 7;
double a[n] = { 10, 11, -3, -2, 0, -1, 4 };
int count = 0;
int i;
for (i = 0; i < n; i++)
{
    if (a[i] < 0)
    {
        count++;
    }
}
double *b = new double [count]; // b зберігатиме від'ємні елементи
int j = 0;                      // індекс у масиві b
for (i = 0; i < n; i++)
{
    if (a[i] < 0)
    {
        b[j] = a[i];
        j++;                    // наступний індекс
    }
}
for (j = 0; j < count; j++)
{
    cout <<  b[j] << ' ';       // виведення
}
delete [] b;                    // звільнення пам'яті
```

3.4 Видалення цифр з рядка

У наведеній нижче програмі здійснюється читання рядка з клавіатури до натискання користувачем клавіші `Enter`. У новий рядок записуються всі символи окрім цифр

```
#include <iostream>
```

```
#include <cstring>


using namespace std;


int main()
{
    char str[80], result[80];
    cin.getline(str, 80);
    int i, k;
    // Потрібно скопіювати strlen(str) + 1 символ, включаючи '\0':
    for (i = 0, k = 0; i <= strlen(str); i++)
    {
        if (str[i] < '0' || str[i] > '9')
        {
            result[k] = str[i];
            k++;
        }
    }
    cout << result;
    return 0;
}
```

Як видно з програмного коду, залишаються всі символи, коди яких не знаходяться в діапазоні кодів десяткових цифр. У кодовій таблиці ці коди розташовані послідовно.

Для читання рядку до кінця застосовано функцію `getline()` об'єкта `cin`. Для читання рядків можна також використовувати операцію >>:

```
cin >> str;
```

В цьому випадку рядок буде прочитано не до кінця, а до першого розділювача (символа пропуску).

4 Вправи для контролю

1. Написати програму, яка обчислює суму додатних елементів масиву дійсних чисел. Замість індексів використати вказівники та адресну арифметику.
2. Прочитати рядок, замінити пропуски на символи підкреслення та вивести результат.
3. Написати програму, яка читає цілі до значення 0 і обчислює добуток цих чисел без останнього нульового значення.
4. Написати програму, яка читає цілі до кінця файлу і обчислює добуток ненульових значень.

5. Написати програму, яка визначає масив чисел і записує в текстовий файл суми елементів з непарними індексами.

5 Контрольні запитання

1. Що таке вказівник?
2. Як отримати адресу змінної?
3. Як здійснюється розіменування?
4. Які є варіанти використання вказівників?
5. У чому різниця між постійним указівником і вказівником на постійний об'єкт?
6. Як отримати адресу масиву?
7. Який є взаємозв'язок між масивами і вказівниками?
8. Що таке адресна арифметика?
9. Що є результатом оператора "мінус", застосованого до двох указівників?
10. Які області комп'ютерної пам'яті використовують для розташування змінних?
11. Що таке динамічна пам'ять?
12. Як розташувати змінну в динамічній пам'яті?
13. Чому необхідно видаляти непотрібні змінні з динамічної пам'яті?
14. Як розташувати двовимірний масив у динамічній пам'яті?
15. У чому є специфіка символьних масивів?
16. Що таке рядок, який закінчується нулем (null-terminated string)?
17. Які функції мови C можна використовувати для виведення і введення?
18. Які є недоліки використання функцій `printf()` і `scanf()`?
19. Чому явне закриття файлів у C++ не є обов'язковим?
20. Як перевірити, що ми дісталися кінця файлу?